IssueForge v1 decomposition — FAILURE REPORT (no issues created)

Status: BLOCKED. The independent review gate rejected the decomposition twice. Per the run protocol, no GitHub issues were created and no resolution was invented. This report records what was found and what decisions are needed to unblock.

Date	2026-07-13
Repository	`MatthewDruhl/IssueForge` (private)
Source PRD	#1 — *PRD: human-gated TDD issue runner*
PRD scale	51 acceptance criteria across 11 user stories
Review 1 (draft v1)	Codex, fresh session → **REVISE** (12 blocking findings)
Review 2 (draft v2)	Codex, fresh session → **REVISE** (7 blocking findings)
Review 3 (option (d) for D1)	Codex, fresh session → **BROKEN** — but it *resolves* D1 (see below)
Review 4 (D2)	Codex, fresh session → **ADD-OVERRIDE** — and it corrects a mistake in Review 2
Review 5 (D3)	Codex, fresh session → **SPLIT-SHAPER**
Child issues created	**0**
Issues modified	**0** (PRD #1 untouched)
MARVIN files/issues modified	**0**
Draft artifact	`/private/tmp/issueforge-prd-to-issues-draft.md` (21 v1 issues + 2 `deferred-v2`)

Executive summary

Two rounds of adversarial review were run against a full 51-criterion decomposition of PRD #1. The first round found 12 blocking defects; all 12 were addressed in a second draft. The second round, run in a fresh session, found 7 further blocking defects.

The protocol for this run states: "*If blocking findings remain after the second review, do not invent a resolution. Create no GitHub issues, generate a failure report PDF, and exit nonzero.*" That is what happened. The gate did its job.

Three of the seven findings are not decomposition bugs — they are conflicts with PRD #1 itself, and they require a human decision, not a cleverer decomposition. That is precisely the situation the no-invented-resolution rule exists to protect. The remaining four are real engineering defects with clear fixes, but they cannot be applied in isolation while the PRD conflicts are open, because two of them change what the affected issues must contain.

The decomposition work was not wasted: the draft, the criterion inventory, the MARVIN source audit, and the review transcripts are the raw material for the next attempt.

Three further adversarial reviews (3, 4, 5) were then run to pressure-test the three blocking decisions. All three now have evidence-backed recommendations, and all three require amending PRD #1. They are not applied here — the protocol forbids inventing resolutions, and a PRD amendment is the author's call, not the decomposer's.

Review 4 also corrected an error in Review 2 that this report had accepted. The PRD *does* grant an implementation-review override (`prd-v1.md:153`); Review 2 asserted otherwise and was believed without checking the source. The review gate is not infallible either, and its claims about the source documents must be verified against the source documents. That correction is recorded in D2 below rather than quietly fixed.

The four decisions needed to unblock

D1 — Is IssueForge v1 pytest-only, or genuinely repository-agnostic? *(blocking, PRD conflict)*

PRD #1 promises operation on "*any explicitly registered local GitHub repository*" (`prd-v1.md:9`; `architecture.md:5`). But every verification concept the PRD relies on is pytest-shaped: "collected identifiers" (US-5.5), the collect/execute distinction (US-5.1), and the AST integrity machinery the architecture points at.

A generic argv baseline command (`npm test`, `go test`, `cargo test`) **cannot** yield a stable collected-node identity set, and without that there is no freezable discovery boundary — so there is no contract to freeze and no integrity gate to enforce.

Draft v2 tried to have it both ways: ship a pytest adapter, plus a `generic` adapter that supports only pass/fail/timeout and **refuses to author a contract**. The reviewer rejected this, correctly:

> *"A successfully registered Go, JavaScript, Rust, or non-pytest Python repository therefore cannot complete the defining v1 workflow. Calling that limitation explicit does not make it PRD-conformant."*

Options:

- **(a) Amend the PRD** to state v1 is pytest-only, and make `repo add` reject a non-pytest repo at registration rather than at contract time. Smallest change; honest; narrows v1.

- **(b) Design a framework-neutral test-identity contract** in v1 (a per-adapter "stable test id + dependency closure" interface, with adapters for pytest and at least one other). Preserves the promise; materially enlarges v1.
- **(c) Keep the generic adapter but define a weaker, explicitly-named contract mode** for it (e.g. whole-suite red/green with no identity freeze), and amend US-5.5/US-6.1 to say what integrity means in that mode.

This decision changes issues #2, #9, #11, and #12. It cannot be deferred.

D1 — RESOLVED by a third adversarial review: take (a), with a refinement

A fourth option was proposed and pressure-tested in a fresh Codex session: a **"discriminates-or-fails"** integrity core. Instead of enumerating what could neutralize a test (which needs framework introspection), *prove the test still discriminates*: at the readiness gate, take HEAD's tree, revert **only** the approved implementation footprint to the contract commit, run the frozen command, and **require the suite to go red and reproduce the recorded evidence.** It needs only git and an exit code, so it would have been framework-neutral — and it appeared to kill the `helpers.py` fixture bypass (finding 2) as a side effect.

Verdict: BROKEN. The review found a working bypass, entirely inside the approved footprint, touching no test, fixture, or config:

```
def authorize_payment(amount: int) -> bool:
    if "PYTEST_CURRENT_TEST" in os.environ:   # pytest sets this during a test run
        return True
    return False                              # the real behavior is never implemented
```

Reverting the implementation restores the *exact* original assertion failure, so the discrimination run passes. At HEAD the suite is green. In production the behavior does not exist. **Both gates pass and nothing was delivered.**

The category error, precisely: the check proves a **counterfactual dependency** ("something in the footprint is necessary to turn red into green"). The PRD requires **behavior delivery** (`prd-v1.md:64` — "*green means the approved behavior was delivered*"). Those are different properties. The attack generalizes beyond pytest (Go's `.test` binary suffix, Jest worker variables, parent-process names, stack-frame inspection).

Two further breaks: **conditional neutralizers** (an out-of-footprint fixture can check for an implementation sentinel and only neutralize when it is present, so reverting the impl restores the original red on cue), and **hermeticity** (a scratch git worktree reverts *tracked files* only — not `node_modules`, the venv, `target/`, build caches, or an already-migrated test database, so honest implementations would be blocked by false positives).

And the D1 claim itself was false. An exit code cannot distinguish a behavioral red from a compile failure, zero tests collected, a skipped suite, a missing dependency, or a timeout — a distinction

`architecture.md:14` explicitly requires. `go test ./...` fails when a package does not compile; Jest can be configured to pass with no tests; Cargo can fail during dependency resolution. The reviewer's summary:

> "*Repository-agnostic orchestration is realistic. Framework-neutral semantic integrity is not.*"

Therefore: resolve D1 as (a), with a refinement. Ship v1 with **pytest as the only supported target**, but keep the workflow engine and the **verification interface** repository-agnostic. The portable seam is an **adapter contract** — *prepare hermetic environment; enumerate approved tests; run selection; report structured execution/failure phases; normalize behavioral evidence; detect zero/skipped/deselected tests* — **not raw process output.** Go/Cargo/Jest adapters implement that interface later; that is an adapter, not a re-architecture. `repo add` rejects a non-pytest target at **registration**, where the user can act on it.

Retain the discrimination run as defense-in-depth, not as the core. It is a targeted counterfactual mutation and it does catch accidental weakening and *unconditional* neutralizers. It is one signal.

Known residual risk, to be carried explicitly rather than assumed away: the test-environment-detection attack **also defeats the AST / node-id / file-hash design.** No static scheme catches an implementation that behaves differently under test. That risk is irreducible at the static layer and must be carried by adversarial code review explicitly instructed to look for test-context behavior, plus mutation testing (#24) and hermetic, separately-provisioned red and green runs.

Review artifact: `/private/tmp/codex-optiond.out` — VERDICT: BROKEN.

D2 — Does implementation code review get a human override? *(blocking; NOT a PRD conflict — an internal inconsistency, see the resolution)*

US-5.4 explicitly grants an override for the **test-contract** review ("reviewer failure may be explicitly overridden… and the override is recorded"). **US-6.3 grants no equivalent override for the implementation code review** — it flatly requires "*an independent code review with no blocking findings*".

Draft v2 gave the implementation review an override anyway, by symmetry. The reviewer flagged this as inventing scope. Either the override is removed (a blocking finding then means the run pauses for a human to fix, with no bypass), or US-6.3 is amended to grant one.

D2 — RESOLVED by a fourth adversarial review: ADD THE OVERRIDE. The premise above was wrong.

Correction, on the record. The PRD *does* grant an implementation-review override — just not in US-6.3. `prd-v1.md:153` (Implementation Decisions): "*Independent test and code reviews require fresh sessions and support explicit recorded fallback or human override.*" Reinforced by `architecture.md`'s Human-gates list — "*an independent AI review must be overridden*" (generic, not test-only) — and by US-7.2, which has the PR report "*AI review verdicts, and overrides*" (plural).

Review 2's finding #3 asserted no such override exists and this report accepted it **without checking the PRD**. That was a verification failure on our side: the review gate is not infallible, and its claims about the source documents must be checked against the source documents. Draft v2's original instinct — that the override belongs there — was correct; it was removed for a bad reason.

D2 is therefore NOT a PRD conflict. It is an internal specification inconsistency: the cross-cutting Implementation Decision grants the override, and the stage-specific acceptance criterion (US-6.3, the enforceable one) omits it. Leaving that contradiction unresolved is worse than either deliberate policy.

The deadlock is real, and park/cancel is not an escape hatch. Trace it: implementation completes → tests, baseline, gates, integrity, scope all pass → the fresh reviewer raises a *false-positive* blocking finding → the two automatic repair attempts (US-6.2) are consumed failing to "fix" a non-problem → the run pauses. Resuming returns to the same unmet US-6.3 gate. Parking (US-2.4) preserves the unmet state and frees the worker; it does not make the run PR-ready. **US-7.1 then forbids opening the PR.** The "the human can just merge it anyway" argument fails: **US-7.3's merge authority is unreachable, because the PR never opens.** The only remaining path — park, push by hand, open a PR outside IssueForge, reconcile the stranded run — is *routing around the product*, not an escape hatch. Without an override, **a probabilistic reviewer holds an unappealable veto over a deterministic workflow.**

Proposed PRD change. Add a criterion after US-6.3:

> *"A blocking implementation-review finding or reviewer execution failure may be overridden only by an authenticated human, after one fresh independent same-provider review attempt. The override applies only to identified AI-review findings at the exact reviewed head commit; it cannot waive contract integrity, acceptance tests, the full baseline, configured quality gates, approved file scope, or deterministic observability and sensitive-data requirements. The permanent audit trail records the human identity, commit, reviewer sessions and verdicts, each overridden finding, supporting rationale, and acknowledged risk. Any subsequent code or contract change invalidates the override. The PR prominently reports the overridden findings and rationale for renewed human consideration before merge."*

And soften US-6.3's absolute language: "*…and an independent code review with no blocking findings except findings explicitly overridden under the following criterion.*"

Non-negotiable properties of the override (these are the friction that stops it becoming the hole every gate leaks through): human-only (never the implementer AI, reviewer AI, engine, or repair loop); **per-finding, never a blanket "ignore review"**; bound to the exact reviewed head sha, invalidated by any subsequent code or contract change; a fresh replacement reviewer session attempted **first**, so a recoverable session failure is distinguished from a conscious acceptance of risk; and **override means "allowed to open the PR", not "erase the finding"** — every overridden finding stays visible in the PR for merge review. It does **not** authorize merge; US-7.3 is unchanged.

Residual risk, named: the failure mode is cultural. Once an override exists, it gets used whenever review is inconvenient. The per-finding rationale, sha binding, permanent audit, and PR disclosure add friction deliberately. They reduce that risk; they do not eliminate it.

Review artifact: `/private/tmp/codex-d2.out` *— RECOMMENDATION: ADD-OVERRIDE.*

D3 — Must shaping (#18) precede contract authoring in the build order? *(blocking, ordering)*

`architecture.md:9` puts shaping at lifecycle step 2, before worktree creation (3) and test authoring (5). Draft v2 ordered shaping *late* (build the pipeline on an already-buildable issue first, with shaping as a pass-through) on the grounds that shaping is the hardest AI surface and shouldn't be built before its consumers exist.

The reviewer's objection is substantive: **#13's readiness gate requires an "approved file scope", and the producer of that footprint is #18.** As drafted, #13 could be declared unblocked without #18 ever landing.

Options: make a *buildable-path* shaping slice a hard prerequisite of contract authoring (keeping only oversized-issue decomposition, #19, late); or keep the late order and explicitly re-home the approved-file-scope producer.

D3 — RESOLVED by a fifth adversarial review: SPLIT THE SHAPER

A "pass-through shaper" is not a real thing. This is the argument that settles it. At readiness the engine must evaluate "approved file scope" (US-6.3) and, with a pass-through shaper, has no approved set to compare against. Only three behaviors are possible, and all three are unacceptable:

1. **Fail closed** — every run pauses for missing scope. The claimed "end-to-end buildable path" does not actually work, so the milestone is a fiction.
2. **Skip the check** — the milestone ships a readiness gate that **cannot enforce US-6.3**.
3. **Approve after seeing the diff** — "*every diff would approve itself.*" Retrospective ratification. This is the deceptive one: the run *looks* compliant while defeating US-6's whole premise (`prd-v1.md:66` — "*AI implementation constrained by the approved tests*").

The same failure hits observability concretely: with no pre-implementation contract naming sensitive fields, an implementer adding retry logic may log request headers. US-6.7 requires excluding "*contract-listed sensitive fields*" — but with no shaped contract, **there is no list to enforce**. And producing the verdict at review time is too late: "*independent implementation review confirms compliance; it should not originate the requirement it is supposed to review.*"

A REQUIREMENTS DEFECT surfaced that nobody had named. The PRD says the shaper owns "*footprint estimation*" (`prd-v1.md:133`) and that an "*unknown expected footprint pauses the run*" (US-3.4), and that readiness requires "*approved file scope*" (US-6.3). **But it never defines the approval transition between them** — who approves the scope, at which human gate, whether it is exact files or path patterns, and how expansion is authorized. Note also that **US-5.5's contract-freeze list conspicuously omits file scope** ("*the exact test commit, file hashes, collected identifiers, dependent fixtures/configuration, command, and red evidence*"). So the claim that scope is "already approved at contract freeze" is **unsupported by the text**. This gap must be closed by an explicit design decision, not read into the PRD.

The resolution: split the shaper. Epic decomposition and footprint estimation "*share a lifecycle label, not an implementation risk profile.*"

New early slice — "Buildability Contract." Outputs: readiness classification (`buildable` / `oversized` / `blocked`); duplicate verdict + evidence; unresolved-design-decision list; the **proposed expected footprint** (allowed files/path patterns + justification) and an explicit `footprint_known` verdict; the **observability verdict** with reviewer-confirmed justification and, when required, the success/failure events and prohibited sensitive fields; and an immutable shape-artifact version. Transitions: duplicate / unresolved decision / unknown footprint → **pause**; `oversized` → stop at `decomposition_required` (**do not pretend this slice can process it**); `buildable` → **the human approves the buildability contract, including file scope, before contract authoring**; later scope expansion → **new human authorization**, preserving the prior approval in the audit trail.

It owns US-3.4 and US-6.5 completely, plus the producer side of US-6.3's approved file scope and the shaped-contract side of US-6.6/6.7 (implementation and review still own *enforcement*). **It does NOT claim US-3.1–US-3.3** — in-place revision, epic decomposition, child drafting, and GitHub mutation stay late, where their downstream consumers (queue, gateway, closeout, idempotency keys) already exist. The late-shaper instinct was right about *those*; it was evasive about footprint and observability.

Recommended build order for the first five issues:

1. **Persistent workflow kernel + artifact interfaces** — run state, transitions, pause/resume/park, approvals, event persistence, fake stage adapters. Foundational parts of US-2 and US-10.
2. **Repository registration + isolated workspace + baseline** — US-1 and US-4. *Built before shaping even though it runs after shaping at runtime.*
3. **Buildability Contract** (the new slice) — US-3.4, US-6.5, producer side of US-6.3.
4. **Acceptance-contract authoring and freeze** — US-5. Consumes the approved shape artifact.
5. **Implementation + integrity + readiness + green PR** — US-6 and the delivery part of US-7.

Then in-place revision, then epic decomposition and child mutation/queueing.

> *"Fail-closed missing artifacts are safer than silently unenforced gates. But the full early shaper wastes risk on decomposition before it is needed. Splitting produces the safer dependency order without front-loading the hardest judgment surface."*

Review artifact: `/private/tmp/codex-d3.out` — RECOMMENDATION: SPLIT-SHAPER.

D4 — Is the retry budget one counter or two? *(non-blocking, recorded for completeness)*

US-6.2 says "*at most two automatic repair attempts.*" MARVIN used **three nested caps** and its provenance warns that one counter hides two different failures (`spec-dev/SKILL.md:170-181`). Draft v2 followed the PRD (the correct call — the PRD governs) and mitigated by making review rounds a separate protocol that does not consume the budget. Flagged here only so the divergence is a decision on the record rather than an accident.

Review 2 — the seven blocking findings

#	Finding	Type
1	**#2 breaks the repository-agnostic promise.** A registered Go/JS/Rust repo cannot complete the v1 workflow.	**PRD conflict → D1**
2	**#11/#12 still do not freeze the complete fixture boundary.** Real bypass below.	Engineering defect
3	**#13 pushes before readiness, contradicting US-7.1**; and invents a code-review override US-6.3 does not grant.	Defect + **PRD conflict → D2**
4	**#16 does not guarantee closing the *exact run issue*** (US-8.2).	Engineering defect
5	**#20's lint cannot establish the completeness US-11.1–11.4 demand.**	Engineering defect
6	**The dependency graph does not enforce shaping before authoring/implementation.**	**Ordering → D3**
7	**#9's "baseline remains green in the same run" is ambiguous and, as written, impossible.**	Engineering defect

Finding 2 — the fixture bypass (the most important engineering defect)

Draft v2's manifest freezes the **test modules'** transitive import closure and hashes `conftest.py`. It does **not** hash **conftest's own transitive dependencies**, pytest plugins, or other adapter-loaded fixture modules.

The bypass, concretely:

1. `conftest.py` imports a fixture implementation from `tests/helpers.py`.
2. The implementer edits **only** `helpers.py`.
3. The `conftest.py` hash is unchanged. Every test-file hash is unchanged. The command is unchanged. The collected node-id set is unchanged.
4. `helpers.py` is not necessarily in the *test modules'* import closure — only in *conftest's*.
5. The fixture now neutralizes the contract, **undetected**.

This violates US-5.5 and US-6.1. The fix is a genuinely **engineering-discovered dependency closure** covering tests, fixture providers, plugins, configuration loaders, **and their transitive repository dependencies** — returned by the adapter, not declared by a user glob. Note this fix is entangled with **D1**: what a "dependency closure" even means is adapter-specific.

Finding 7 — the baseline-green contract is impossible as written

Issue #9 requires "*the preexisting baseline stays green in the same run*" — the check that catches an author who breaks shared setup while writing the new tests.

But **running the repository's ordinary baseline command at the test commit will include the new, intentionally-failing acceptance tests, and therefore be red by construction.** And simply reusing #6's

earlier baseline result cannot detect author-introduced conftest/config breakage, which is the entire point of the check.

The fix: the adapter needs an operation that executes the **preexisting test-ID set at the contract candidate, excluding only the newly authored acceptance IDs.** That is a new adapter capability, and again it is entangled with **D1** (a generic adapter has no test IDs to exclude).

Finding 4 — closeout can miss the run issue

US-8.2 requires closing "*the exact run issue.*" Draft v2's #16 closes only formal `closingIssuesReferences` (inherited from MARVIN, where that scoping rule is a genuine safeguard against closing unrelated issues).

Two failure modes: a PR with **no** closing reference leaves the run issue **open**; a PR with **multiple** closing references can close issues **other than** the run issue.

Fix: closeout must operate on the **persisted, repository-qualified run-issue identity**. Closing references may be verified or reported, but cannot substitute for that identity. (The MARVIN safeguard is still worth keeping *as a bound* — close the run issue, and never close anything beyond the linked set.)

Finding 5 — the source-audit lint cannot prove completeness

US-11.1 demands an inventory of *every* corresponding MARVIN skill, script, test suite, and failure-driven update. #20's lint checks against `architecture.md`'s "Initial source map" and the transfer ledger's "*including at minimum*" list — **both of which are expressly non-exhaustive.** A record can therefore look complete while omitting an unlisted canonical skill or supporting test.

A real mechanism needs a **versioned authoritative inventory** of canonical source artifacts and behavior/test identifiers, failing on: unclassified entries; extract/refactor decisions with no source-test disposition; reused safeguards with no mapped ported tests; and discovered canonical artifacts absent from the inventory.

What review 2 confirmed as FIXED from review 1

Recorded so the next attempt does not re-litigate settled ground.

- **US-9.2 — all eight TUI views.** Fixed. Draft v1 had silently weakened it ("logs and diffs may ship thin"); v2 requires all eight and depends on their producers.
- **Deterministic vs semantic meaningful-red.** The #9/#10 boundary is now "*conceptually correct*": collection identity, execution phase, baseline health, XPASS, and SHA binding are deterministic; **correspondence with the named behavioral reason is semantic** and belongs to the AI reviewer and the human approver. (Draft v1 wrongly claimed determinism could prove the *named reason*.)
- **Dependency-hash comparison at head.** Added correctly — *but the discovered set is still incomplete* (finding 2).

- **Delete-safety (#15) and cleanup independence (#17).** Substantially fixed. Delete-safety is now bound to the exact merge commit and recorded head SHA; cleanup is an independent stage result, so a post-merge health failure no longer vetoes independently-safe cleanup.
- **The no-MARVIN-write-back boundary (#23).** "*Substantially fixed… the sandbox lifecycle, write monitor, source scan, read interfaces, and permanent CI execution are credible.*"
- **Deferring mutation testing (#24) and the invariant lens (#25) to v2.** Explicitly endorsed: "*Deferring mutation testing is defensible… US-5's meaningful red can be achieved through deterministic red proof plus independent semantic coverage review. Mutation is not textually required by the PRD.*"

Recommended first vertical slice (unchanged, and still sound)

Both reviews accepted this. The first tracer bullet with **no AI in it at all**:

`repo add` → enqueue → fetch → isolated worktree → run baseline → pause.

It exercises the subprocess seam, config, run store, registry, workspace, and verification runner with fully deterministic tests and zero provider dependency, and it de-risks every seam the rest of the system sits on. The smallest demoable unit inside it is `issueforge repo add DandD:~/Projects/DandD && issueforge repo list`.

Review 2's caveat is fair and worth carrying: this is *infrastructure* validation, not yet a complete **product-lifecycle** tracer, because a buildable shaping pass is not in it (see **D3**).

Findings worth keeping regardless of how the decisions land

These came out of the MARVIN source audit and survive any decomposition.

1. **The load-bearing control has no prior art.** `check_acceptance_integrity.py` imports only `argparse, ast, sys, pathlib` — it never runs pytest, never collects, never executes; it diffs syntax trees. "*Verify it is red today*" exists as prose in exactly one place (`spec-wave/SKILL.md:137`). The meaningful-red predicate is **net-new. Nothing to port.** It therefore *looks smaller than it is*, and any plan that files "port MARVIN's guards" as its integrity slice ships a gate that accepts **any** failure as red.

2. **The zero-collected false-read is live in MARVIN today — verified first-hand.** `merged_runner._parse_pytest_summary` (`:681–693`) regexes `N passed` / `N failed` from stdout and flags `red-main` when `failed > 0 or returncode != 0`. It cannot see pytest's exit 5 (no tests ran), a collection error, or an XPASS. **On 2026-07-12 a `/merged` run against DandD reported `red-main` with `passed: 0, failed: 0`** — a suite that collected *nothing*, misreported as a red suite. Zero-collected is a **third state: broken.**

3. **IssueForge's integrity gate can be strictly stronger than MARVIN's.** MARVIN needs a sanctioned exception to its protected-path gate because its implementer removes the PENDING marker (the

Step-4 flip). IssueForge drops PENDING-on-main entirely, so there is no flip and **no carve-out is needed** — the protected-path diff gate can be *absolute*. Dropping that convention also dissolves an unresolved MARVIN contradiction (Phase-2 OQ1: the implementer edits the acceptance file to remove the marker, which directly contradicts "the suite is physically outside the implementer's write scope" — both cannot hold).

4. **A failed read is never negative evidence.** `merged_runner.py` hand-codes this at five call sites, each comment marking a shipped bug: failed `gh pr view` ≠ unmerged; failed `git status` ≠ clean (reading it as clean *discards uncommitted work*); failed `git worktree list` ≠ no worktree; `merge-base` exit 128 ≠ unreachable (it errors on a **genuine clean merge** until you fetch — read naively, a correct merge looks like a *stranding*); `ls-remote` exit 128 ≠ absent. This must be structural (a type), not re-derived per call site.

5. **Empty AI output is not a clean review.** A bare `codex exec` blocks on stdin without a TTY and hangs forever; `2>/dev/null` makes "hung" and "failed" indistinguishable; and **an empty response read as a clean review is a false PASS on the only gate between an AI's work and a human's merge.** Also: `codex exec` has **no network** — `gh` calls inside it stall forever, so the review packet must be materialized to local disk.

6. **Every gate needs a legitimate escape hatch, or people route around it.** MARVIN's own evaluation: "*The amendment path is unrealistic, so amendments route around it.*" Build the amendment path **with** the integrity gate, not after people start bypassing it.

Process notes

- **Discovery** ran as four read-only agents in parallel (PRD coverage; MARVIN provenance; IssueForge architecture; delivery & safety). All four completed. None wrote any file. MARVIN was treated as read-only provenance throughout.
- **The review gate was itself run under the guarded-launch contract** the decomposition mandates: stdin closed, stderr captured to a file (never `2>/dev/null`), a wall-clock timeout, full output persisted, and empty-output-or-nonzero treated as FAILED. This caught a real failure: the first launch exited 127 (`timeout` is not present on macOS) with empty stdout. Had stderr been discarded, that would have been indistinguishable from an empty review — i.e. a false pass. The guard worked.
- **Review artifacts:** `/private/tmp/codex-review-1.out` (+`.err`), `/private/tmp/codex-review-2.out` (+`.err`).

Verification

- No GitHub issues were created, edited, labeled, commented on, or closed in `MatthewDruhl/IssueForge`. PRD #1 is **unmodified**.
- Labels were created in `MatthewDruhl/IssueForge` in preparation (`epic`, `v1`, `deferred-v2`, `phase:0` – `phase:5`, `route:spec-up`, `route:direct-tdd`). They are unused and harmless; delete

them if the next attempt chooses a different taxonomy.

- **No MARVIN file, state, skill, ledger, configuration, generated artifact, or GitHub issue was modified.**
- No IssueForge source code or tests were changed. No implementation branches or PRs were created.

PDF verification. `issueforge-v1-decomposition-report.pdf` — **12 pages, 0 blank pages**, all pages rendered and inspected. Two rendering defects were caught on the first generation and fixed before this version: a paragraph beginning `#9 …` was parsed as a lazy `<h1>` and rendered as a broken headline, and the D1 `Options:` list collapsed into an inline run of dashes. Both are corrected; a regression guard now asserts the document contains exactly one `<h1>` (the title). Tables, code spans, blockquotes, and links render correctly; no clipped text.

Exact next recommended command

All three blocking decisions now have evidence-backed recommendations (reviews 3, 4, 5). Each still needs your sign-off, because **all three require amending PRD #1**:

- **D1 → (a) + refinement.** v1 supports **pytest targets only**; the engine and the *verification interface* stay repository-agnostic (an adapter contract, not raw process output). `repo add` rejects a non-pytest target at registration.
- **D2 → add the override.** Reconcile US-6.3 with `prd-v1.md:153`, which already grants it. Human-only, per-finding, sha-bound, reported in the PR. **This corrects an error in Review 2 that this report had accepted.**
- **D3 → split the shaper.** A new early **Buildability Contract** slice (US-3.4, US-6.5, producer of US-6.3's approved file scope) lands before contract authoring. Epic decomposition and in-place revision stay late. **Also close the newly-found requirements defect:** the PRD never defines how the *expected footprint* becomes the *approved file scope*.

Once the PRD is amended:

```
/prd-to-issues for MatthewDruhl/IssueForge#1
```

re-run against a PRD amended per D1–D3 (or against explicit written answers to them), reusing the draft at `/private/tmp/issueforge-prd-to-issues-draft.md` as the starting point rather than starting from zero.